EECS708: Machine Learning

Deep Learning

Dr. Ioannis Patras
School of EECS

Slide thanks: Dr. Stavros Petridis, Dr. Georgios Tzimiropulos

Course Context

- Supervised Learning
  - (Linear) regression
  - Logistic Regression (Classification)
  - Neural Networks
- Unsupervised
  - Clustering
  - Density Estimation
  - Dimensionality reduction (partial)
- Advanced topics
  - Deep Learning, Convolutional Neural Networks
  - Ensemble Learning

Overview

- Mini-Batch Gradient Descent (SGD)
- Optimisation (momentum, learning rate, regularization, augmentation)
- Convolutional Neural Networks
  - Convolution and max-pooling
  - AlexNet, VGG, ResNet, MobileNet, Fully Convolutional
- Other topics
  - Transfer learning, Multi-task learning, Unsupervised learning

Batch gradient descent

- All N examples are fed to the network. **Weights are updated** only after **all examples** have been presented to the network (1 epoch of training).

- The batch-size is B = N. There are N/B = 1 iterations per epoch

- For each example: for each weight $\mathbf{w}_i$ the corresponding gradient is computed. The weight $\mathbf{w}_i$ is updated (by $\Delta\mathbf{w}_i$) based on the average gradient over N examples.

- Weights are updated only per epoch

Mini-Batch Gradient Descent

- Mini-Batch: M randomly examples are fed to the network.
  – Batch-size = M (M usually 32, 64, …, 512)
  – M/B iterations per epoch

- For each example: for each weight $\mathbf{w}_i$ the corresponding gradient is computed. The weights are updated (by $\Delta\mathbf{w}_i$) based on the average gradient over M examples.

- Weights are updated M/B per epoch. Updates are **noisy.** This turns out to be good for training!

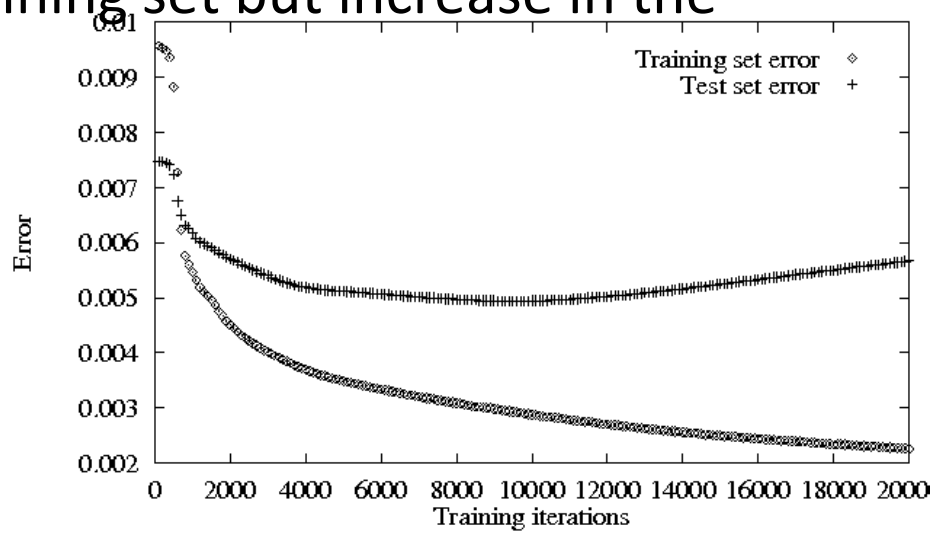
- This algorithm is called **Stochastic Gradient Descent**

Backprop stopping criteria

- When the gradient magnitude (or $\Delta\mathbf{w}$) is small, i.e. $\sum_i |\Delta \mathrm{w_i}| < \epsilon$

- When the maximum number of epochs has been reached

- When the error on the validation set does not decrease
  - Error might decrease in the training set but increase in the validation set (overfitting!)

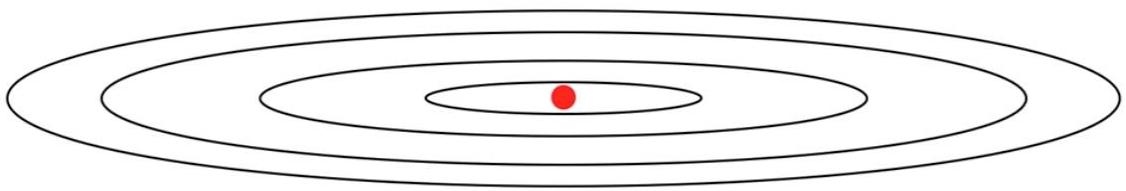


Overview

- Mini-Batch Gradient Descent (SGD)
- Optimisation (momentum, learning rate, regularization, augmentation)
- Convolutional Neural Networks
  - Convolution and max-pooling
  - AlexNet, VGG, ResNet, MobileNet, Fully Convolutional
- Other topics
  - Transfer learning, Multi-task learning, Unsupervised learning

SGD with momentum

- Standard SGD $w_i \leftarrow w_i + \Delta w_i \quad \Delta w_i = -\eta \frac{\partial E}{\partial w_i}$
- Problem with standard SGD: Same learning rate across axes



- SGD with momentum

  μ =0.9

$$\Delta w_i(t) = \mu\, \Delta w_i(t-1) + (1-\mu)\left(\frac{\partial E}{\partial w_i(t)}\right)$$

$$w_i \leftarrow w_i - \eta \Delta w_i(t)$$

- Other algorithms: RMSProp, Adam

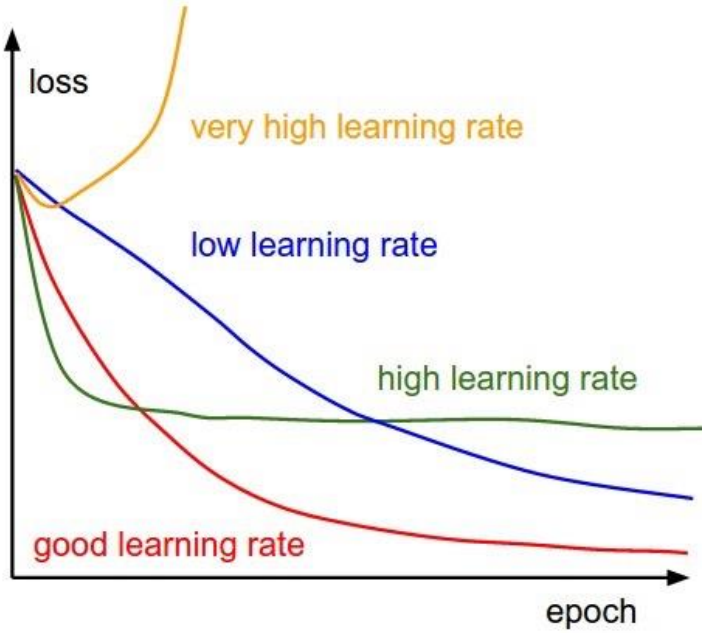
Learning rate decay

- Start with a high (depends on the problem/dataset) learning rate and decay it slowly.
- Typical values for initial learning rate, 0.1, 0.01 (for SGD)
- Step decay: Reduce the learning rate by some factor every few epochs, e.g., divide by 2 every 50 epochs
  – Use a validation set to find when to decrease
- Keep learning rate constant for T epochs and then decrease as follows:

$$lr_t = lr_{t-1} * scalingFactor \text{ (e.g. } 0.99)$$

- You can think your own way too!

Learning rate decay



- If loss increases the learning rate is too high
- If loss goes down slowly the learning rate is low
- Adjust also learning rate based on validation accuracy
- Start with higher learning rate and reduce after several epochs.

Weight Initialisation

Common weight initialisation techniques

- Gaussian distribution with mean = 0, var = 0.02

- Xavier: Gaussian distribution with mean = 0, var = $\sqrt{2/(n1+n2)}$
  – *n1, n2* are the number of neurons in the previous and next layers, respectively.
  - Glorot, Bengio, Understanding the difficulty of training deep feedforward neural networks, JMLR, 2010

- He Initialisation: Gaussian distribution with mean = 0, var = $\sqrt{2/n1}$
  – *n1 is* the number of neurons in the previous layer.
  - Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification, ICCV 2015

- There are many variants of the above
  – (E.g. Gaussian could be replaced by Uniform)

Regularization

Regularisation is a standard technique to reduce overfitting in Machine Learning. Regularisation techniques for NNs include:

- Early Stopping
- L2 regularization (weight decay)
- Dropout
- Augmentation
- More exotic ones (e.g. mix-up)

L2 Regularization

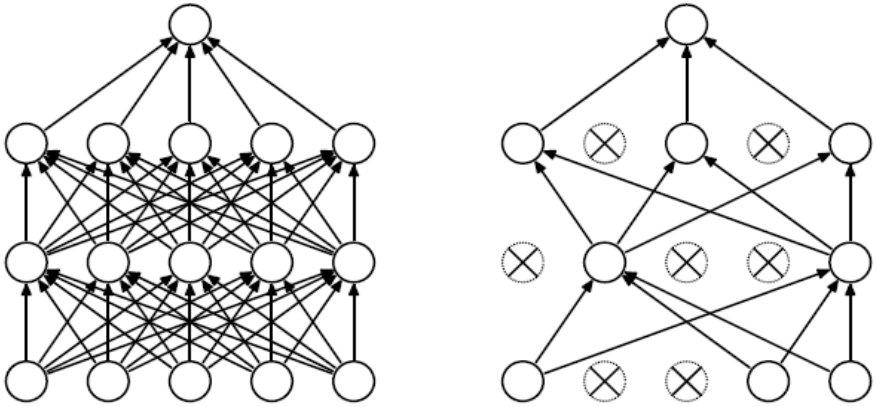
- Large weights dominate the network's prediction
  - because they are tuned to training set, this could lead to overfitting
- Better if all weights have "equal" contribution

$$E = E_0 + 0.5 * \lambda \sum_{all\ Weights} w^2$$

- $E0$ is the original loss function, e.g., L2 loss, CE
- We wish to minimise the original error function $E0$
- We also wish to penalise large weights, keep the weights small (second term)
  - Small λ: we prefer to minimise $E0$
  - Large λ: we prefer small weights
- This is also called **weight decay**

Dropout

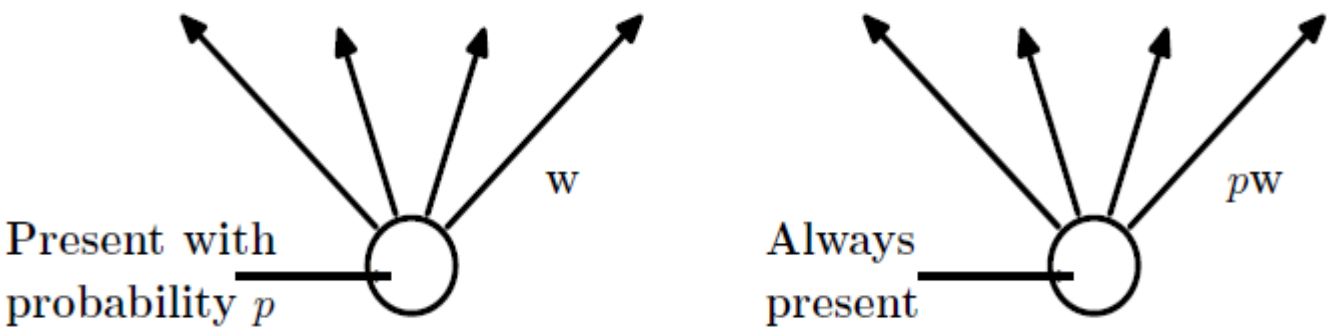
- We don't modify the error function but the network itself
- During training neurons are randomly dropped out
  - Srivastava et al. A simple way to prevent neural networks from overfitting, JMLR 2014
- The probability that a neuron is "preserved" is p



(a) Standard Neural Net (b) After applying dropout.

Dropout

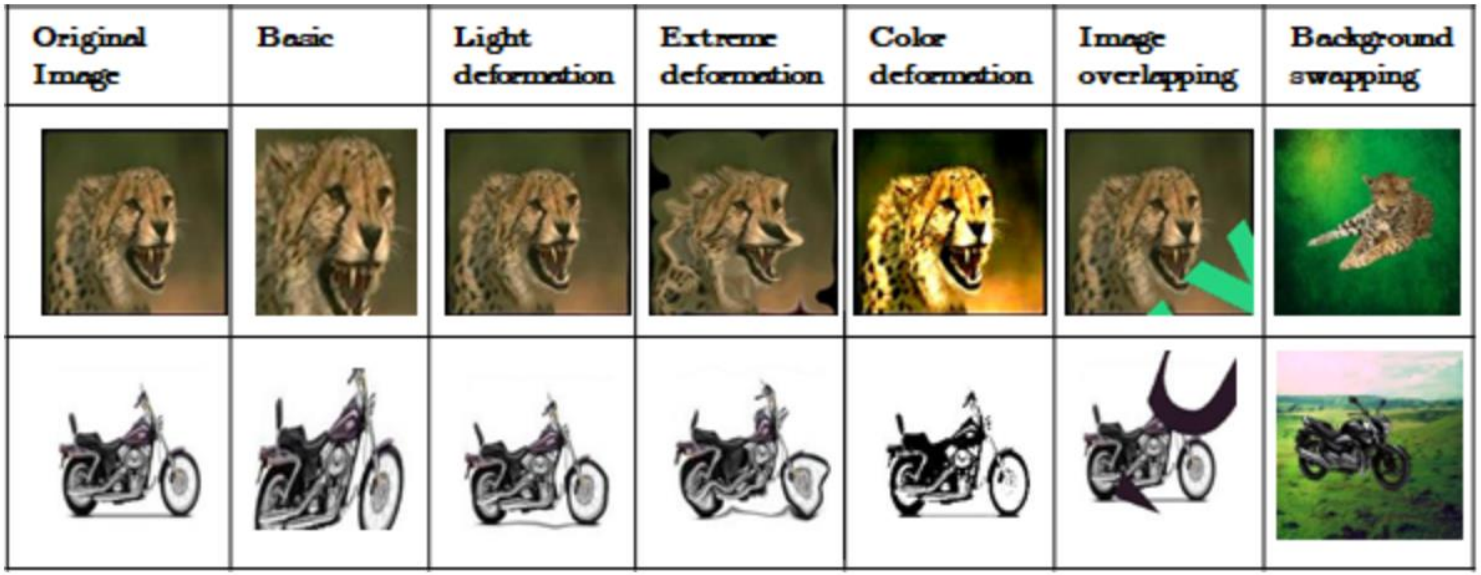
- Dropout prevents overfitting because it prevents neurons from co-adapting too much.
- Each neuron should create useful features on its own without relying on other hidden units to correct its mistakes.
- Test time: outgoing weights of a neuron are multiplied by p.



(a) At training time (b) At test time

Data augmentation

- One of the best ways to avoid overfitting is more data
- We can artificially generate more data by introducing artificially more variations.
- For images: flip left-right, colour, rotate, random cropping, etc.
- At each batch: sample data, apply random augmentation, backprop



Summary

- Mini-Batch Gradient Descent (SGD) introduces noise that is beneficial for learning.

- Optimisation tricks are crucial.
  - Momentum, Adam are often used. Often simple SGD works well.
  - Set learning rate by monitoring the cost. Scheduling of learning rate
  - Regularization is typically used. Dropout often.
  - Data augmentation essential

Overview

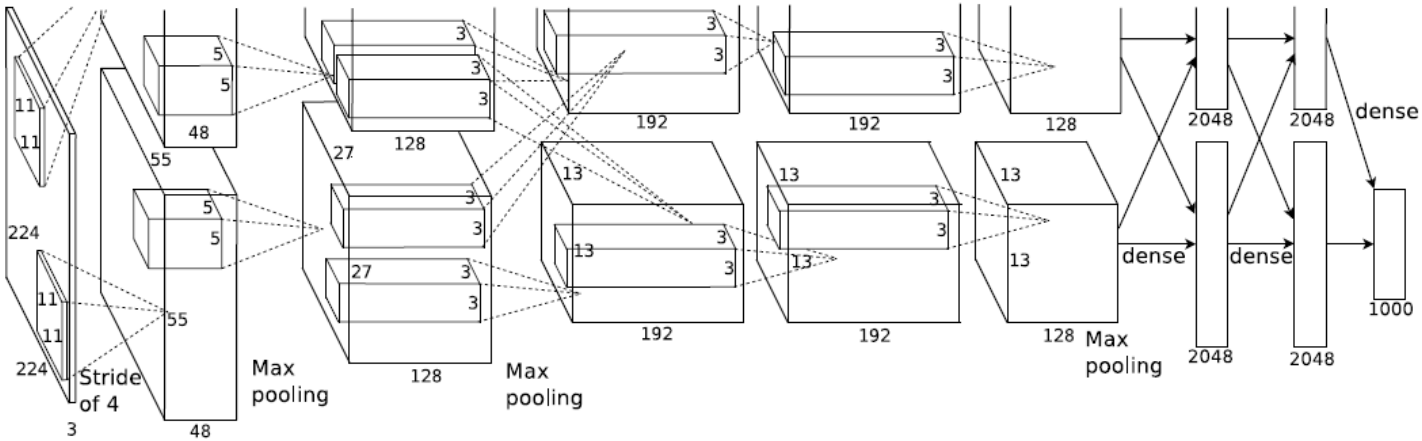
- Mini-Batch Gradient Descent (SGD)
- Optimisation (momentum, learning rate, regularization, augmentation)
- Convolutional Neural Networks
  - Convolution and max-pooling
  - AlexNet, VGG, ResNet, MobileNet, Fully Convolutional
- Other topics
  - Transfer learning, Multi-task learning, Unsupervised learning

Convolutional Neural Networks

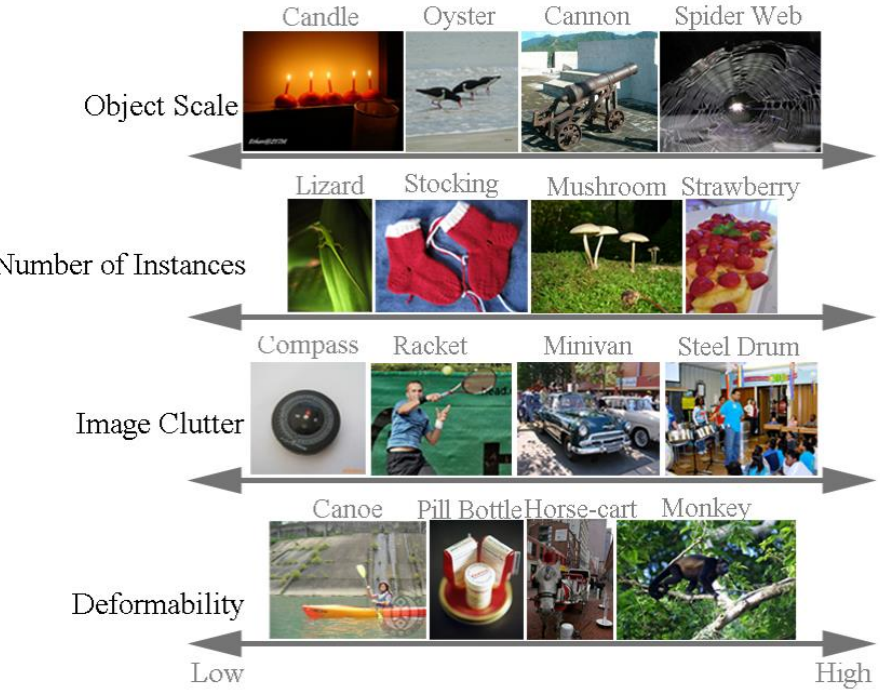
- Convolutional Neural Networks (CNNs) have been very successful in computer vision
- Invented by LeCun in the 80s
- Reference paper: "Gradient-Based Learning Applied to Document Recognition", Proc. IEEE, 1998
- The method of choice for extracting features from images, video and audio

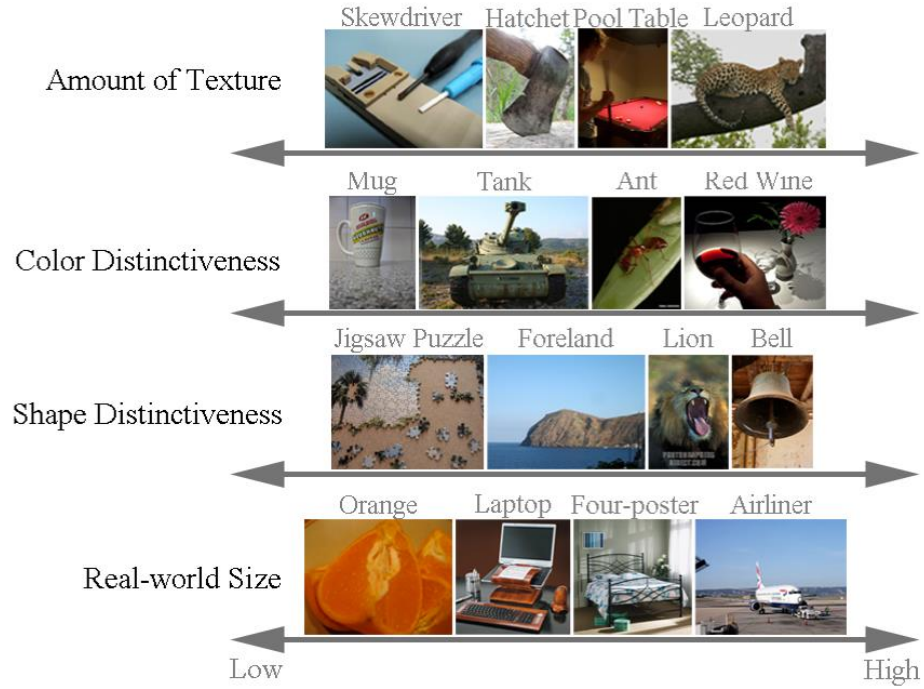
Convolutional Neural Networks

- Image classification has been one of the most difficult problems in AI
- Computer Vision researchers have shown contempt for CNNs
- This paper changed everything: Krizhevsky, Sutskever & Hinton: “ImageNet Classification with Deep Convolutional Neural Networks”, NIPS 2012



ImageNet Competition – Object Classification

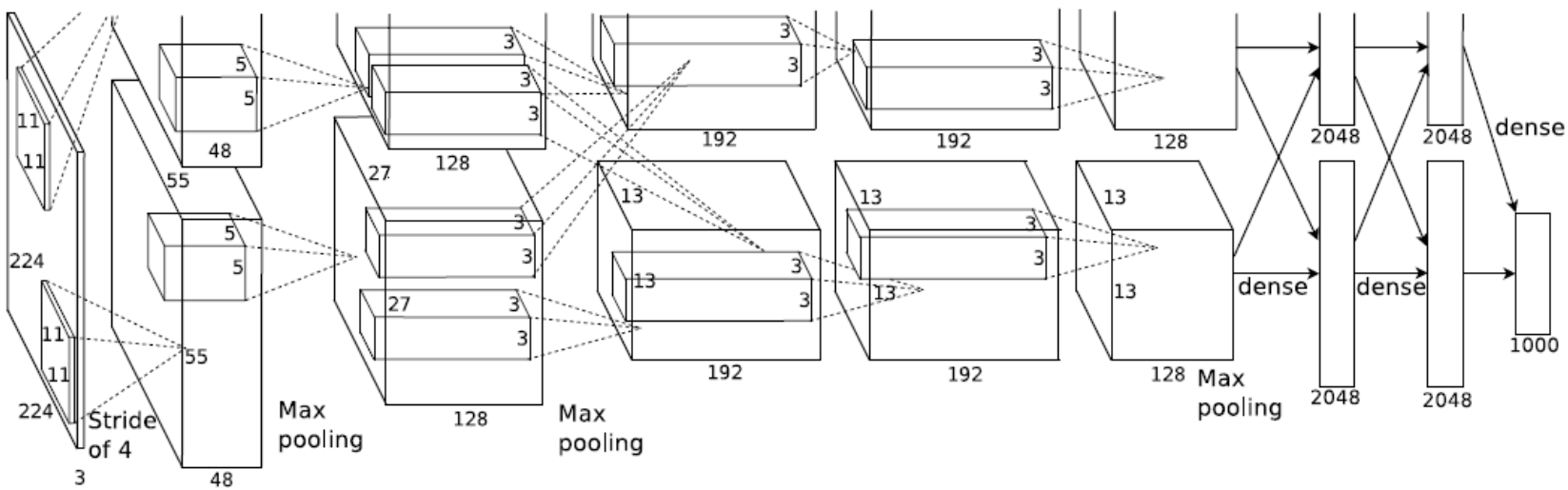




- Classification of 1000+ objects
- State-of-the-art before 2012: error rate ~26%
- New state-of-the-art in 2012 with deep networks: error ~15%
- Tricks: Large dataset, GPUs for training, Augmentation, Dropout, ReLU

AlexNet

- It's a deep network = many layers
- Each layer is either a convolutional layer or subsampling layer
- Final layers are fully connected layers



Convolution

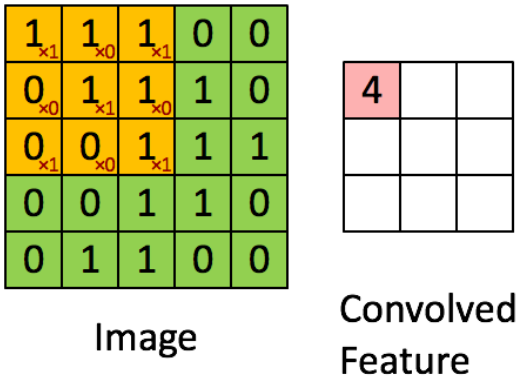
Motivation

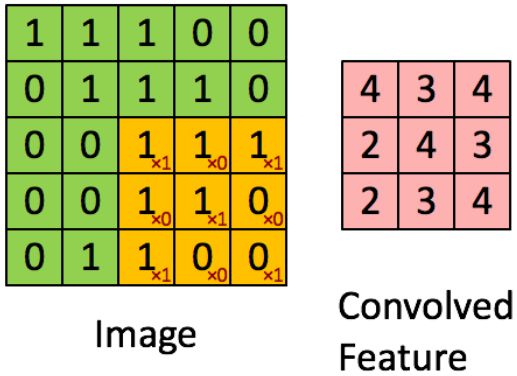
- A standard —also called Fully Connected (FC) — layer is not suitable for images
- E.g.: Assume a grayscale image (1 channel) of resolution 256x256
  - A single neuron connected to all pixels will require 256^2 = 65K params
  - If first layer: 128 of those neurons, in total 65*128= 9M params !
- Convolution Layer:
  - Divide the image into 3x3 windows.
  - Apply *the same* FC layer for all windows
  - A single neuron will have 3x3 params,

  and the whole layer 1K params!

Convolution

- Convolution
  - From:http://deeplearning.stanford.edu/wiki/index.php/Feature_extraction_using_convolution

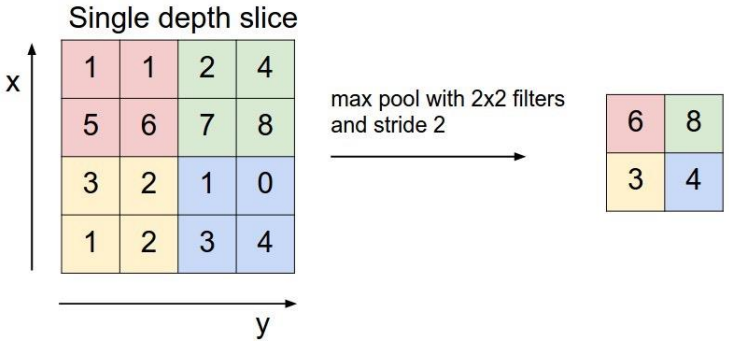




1	**0**	**1**
0	**1**	**0**
0	**0**	**1**

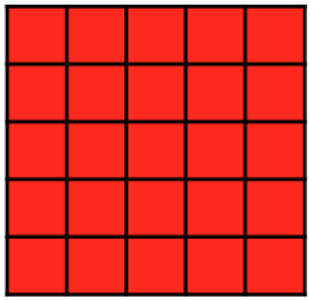
Filter

- Max-pooling

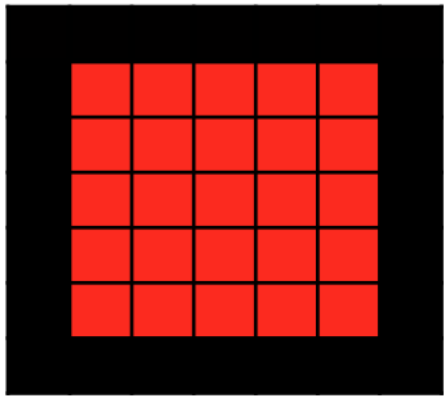


Convolution - 1 input channel

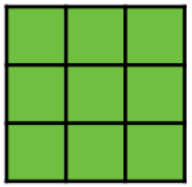
Image

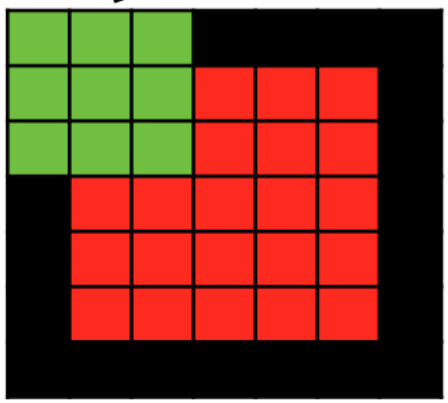


Zero-padded Image



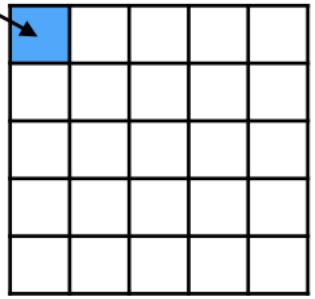
Neuron (Filter)





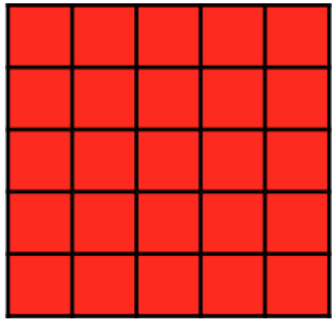
- Multiply each value in neuron with corresponding value from Image patch
- Sum over values

Feature map

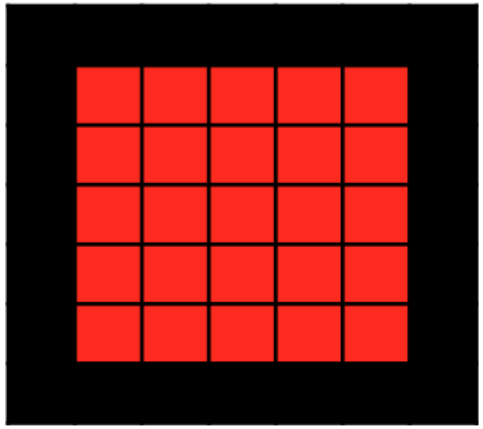


Convolution - 1 input channel

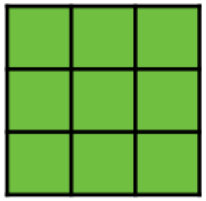
Image

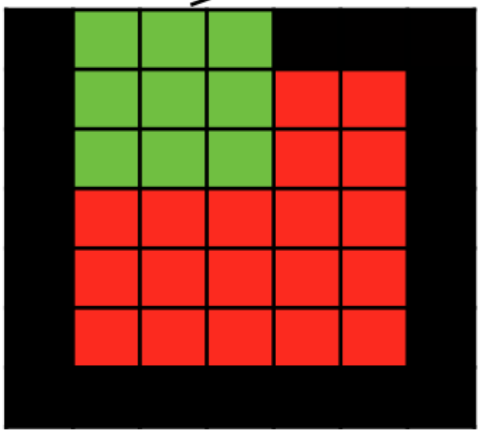


Zero-padded Image



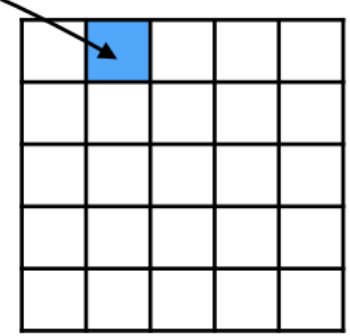
Neuron (Filter)





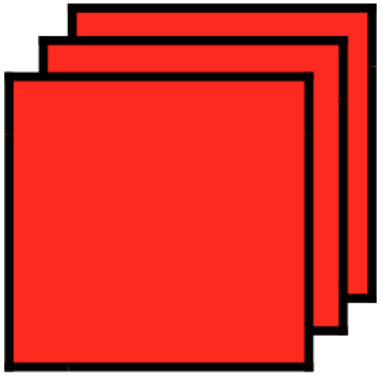
- Shift neuron by one pixel and repeat to get the next value in the feature map

Feature map

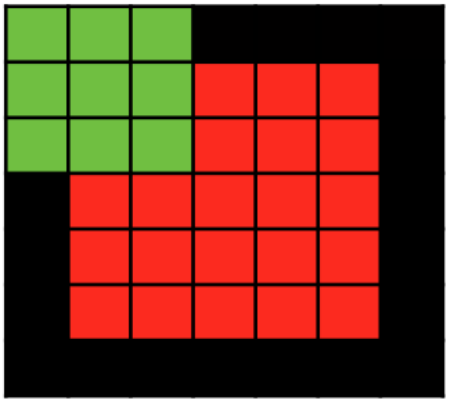


Convolution - 3 input channels

Image

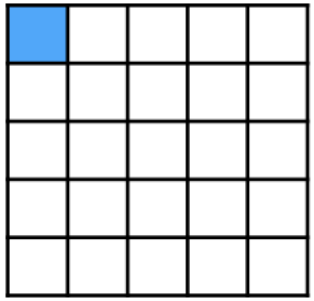


Neuron (Filter)

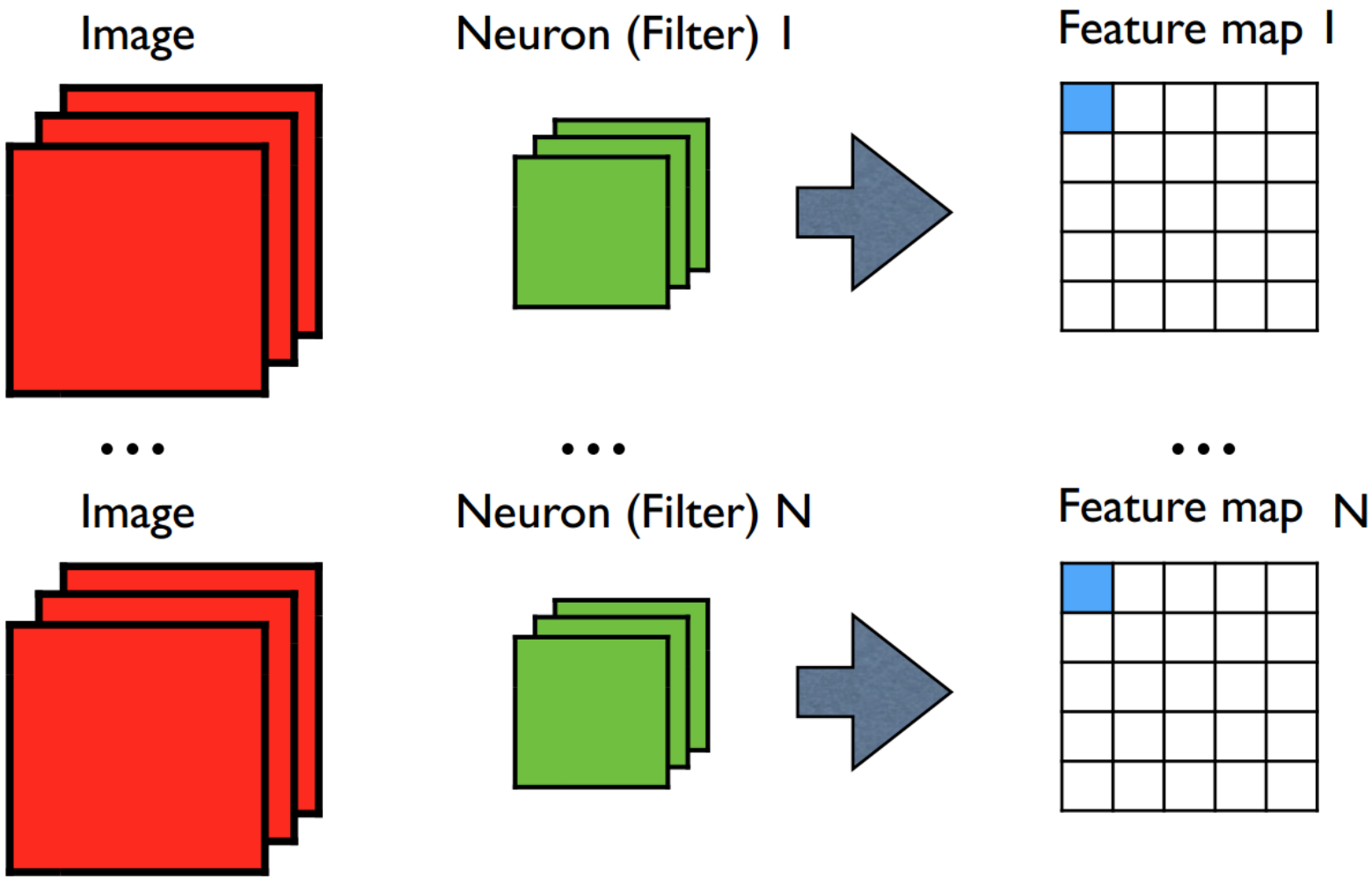


- Same process per channel
  - One 'intermediate' feature map per channel
- Final feature map:
  - sum over all channels
  - again a 2D matrix!

Feature map



Convolution - N output channels

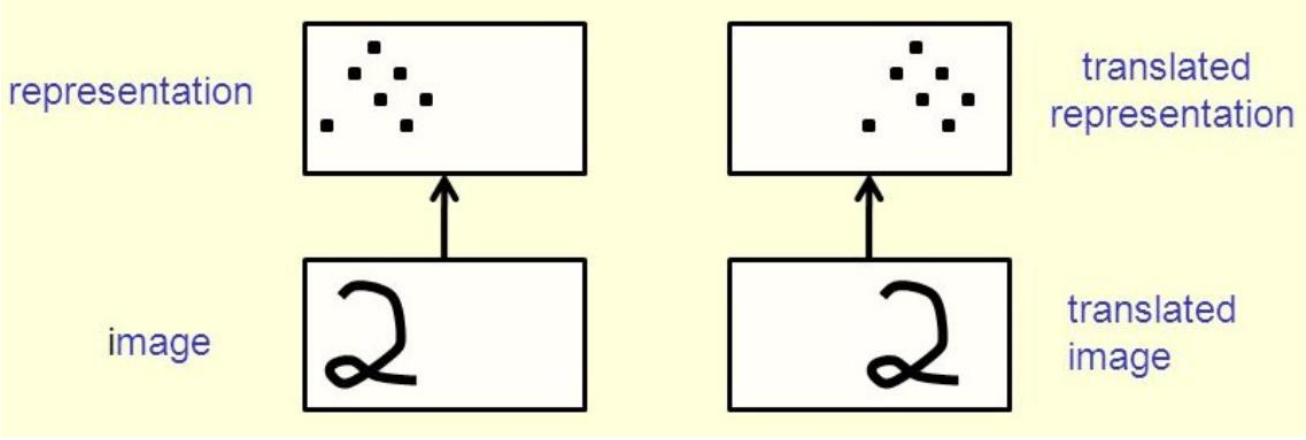


Input to next layer is a stack of N feature maps!

Convolution

Why this works

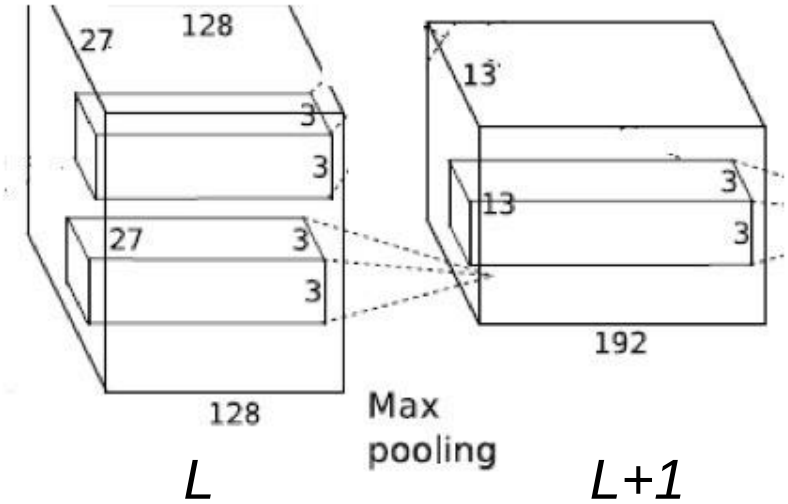
- Image patches have similar statistical properties: we can analyse all of them using the same neuron (filter).
- CNNs have the equivariance property: if an object is shifted by some amount this results in a “shifted” output representation
  - This makes a lot of sense for images (but also audio!)
- Not the case of FC layers



Convolution

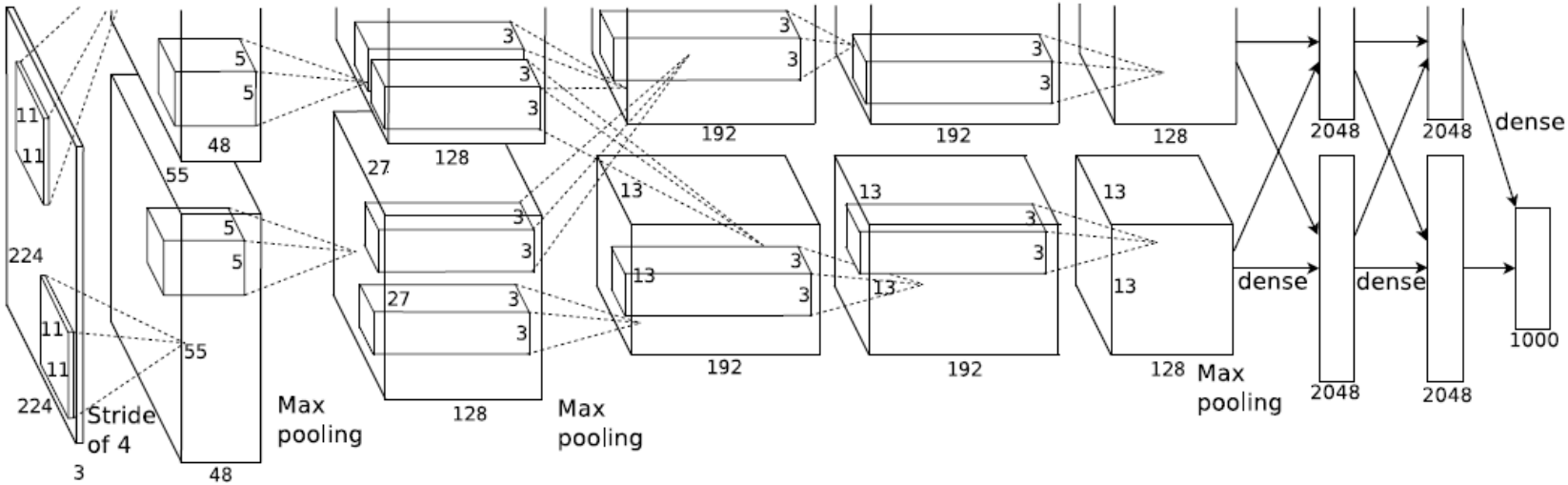
An example from AlexNet

- Input layer has spatial resolution 27x27 and 128 channels (features).
  - If input image layer is RGB: 3 channels
- The number of channels (128) is equal to the number of neurons of the layer L-1.
- A neuron from layer L to L+1 operates on 3x3 patches with 128 channels. So it has 3x3x128 weights
- Layer L+1 has 192 channels: hence, at layer L there are 192 neurons with 3x3x128 params each
- Each of them "scans" the input producing a feature map of 27x27
- Because of max-pooling this is reduced to resolution 13x13
- Since there are 192 neurons at layer L, there are 192 feature maps at L+1



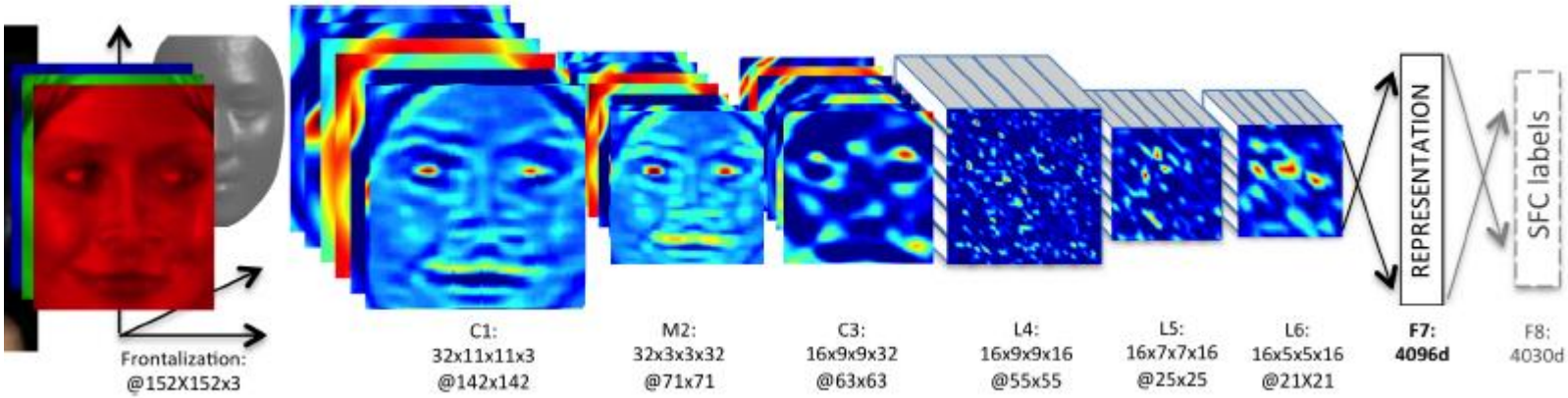
AlexNet — in more detail

- 5 Convolutional layers and 3 Fully Connected layers
- ReLU is applied after every Conv and FC layer.
- Dropout is applied before the first and the second FC connected layer.
- 90 epochs, SGD with momentum, wd=.0005, LR = 0.01— divided by 10 3 times when validation accuracy plateaus



CNNs for face recognition

- One of the most known applications of Computer Vision is face recognition
- With CNNs researchers reporting super-human accuracy

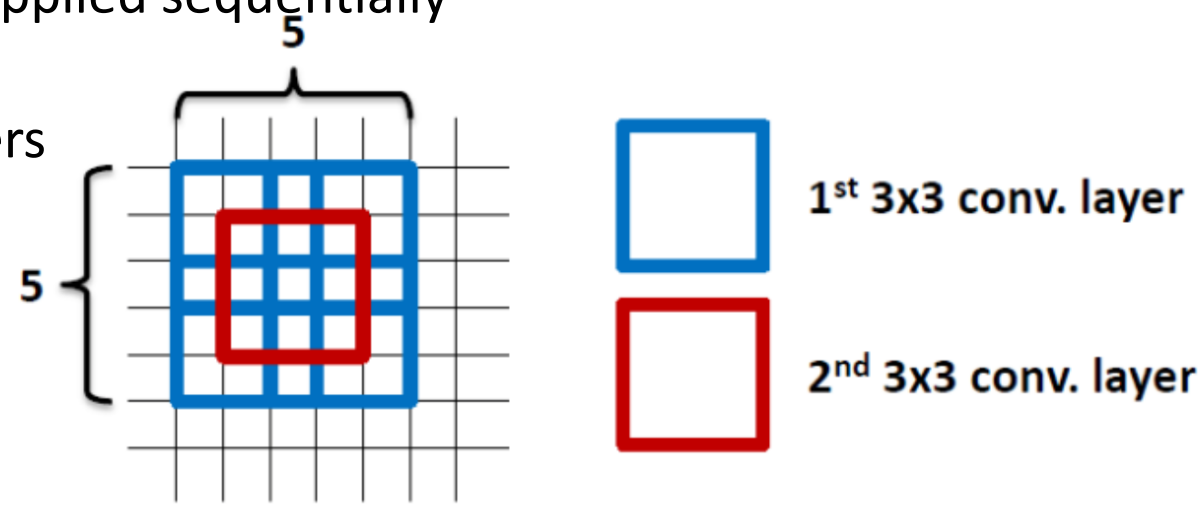


VGG-Net

- An extension to AlexNet which reduces the number of parameters
  - This facilitates training
- Main idea: replace large kernels, e.g. 5x5 or 7x7 with a sequence of small ones
- For example one can show that a 5x5 convolution is equivalent to two 3x3 convolutions applied sequentially
  - 5x5: 25C parameters

  vs 2 3x3: 18C parameters
  - larger reductions for

  larger kernels (e.g. 7x7)

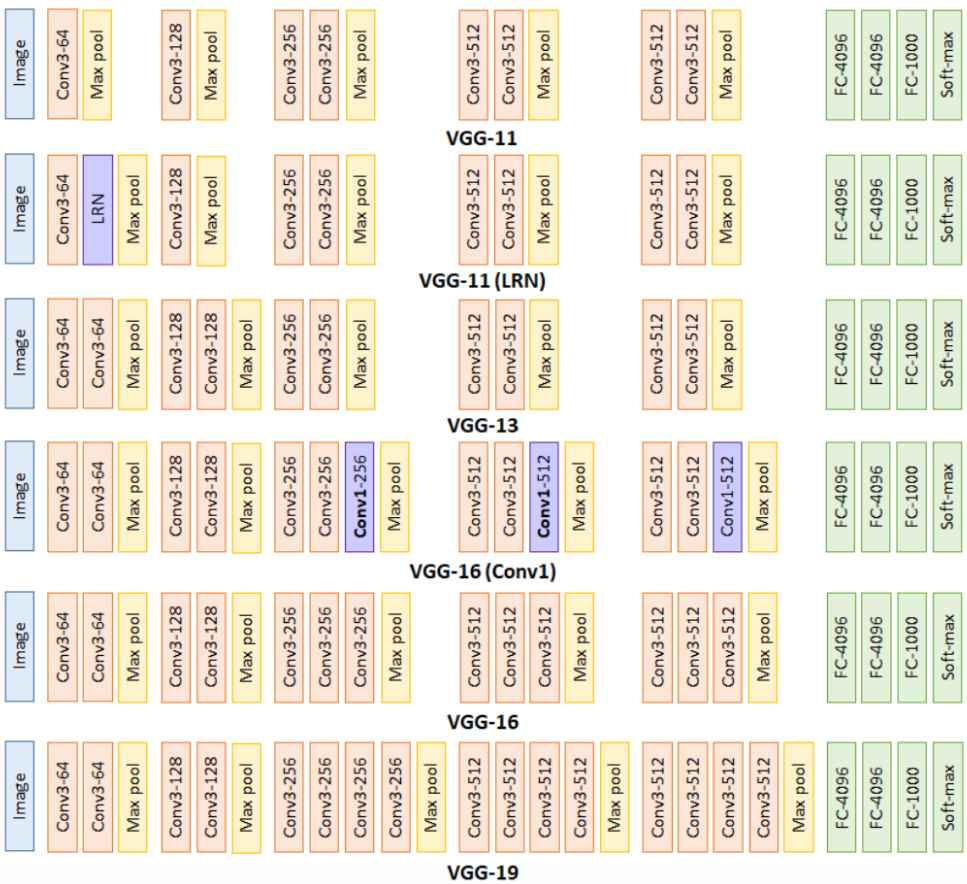


2 layers of 3×3 filters already covered the 5×5 area

Very Deep Convolutional Networks for Large-Scale Image Recognition

VGG-Net

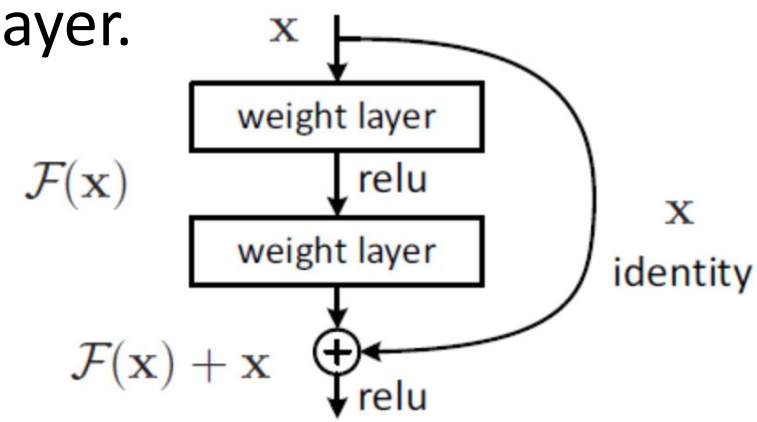
- This enabled training deep networks (up to 19 layers)



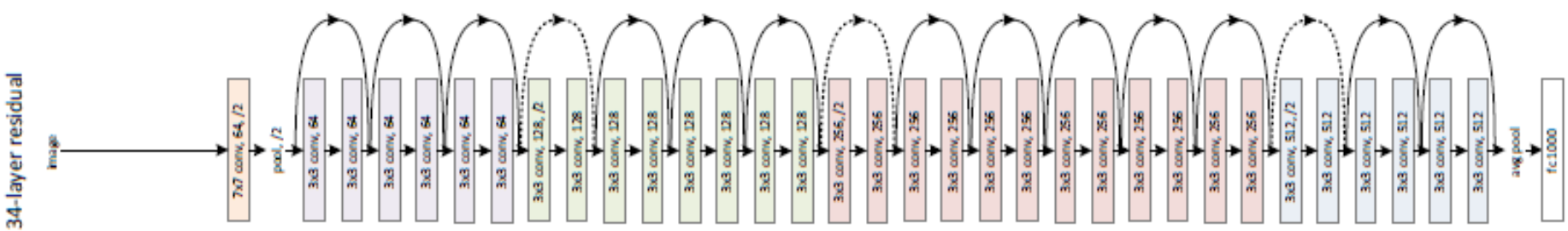
	Number of Parameters (millions)	Top-5 Error Rate (%)
VGG-11	133	10.4
VGG-11 (LRN)	133	10.5
VGG-13	133	9.9
VGG-16 (Conv1)	134	9.4
VGG-16	138	8.8
VGG-19	144	9.0

Res-Net

- A simple method for training very deep networks
  - This facilitates learning very powerful networks
  - Deep residual learning for image recognition
- Main idea: add the input feature to the layer's output
  - output = *F*(input) + input; *F*: conv layer.
  - called **skip connection**
- The advantage is that there's a direct path for propagating gradients during back-prop

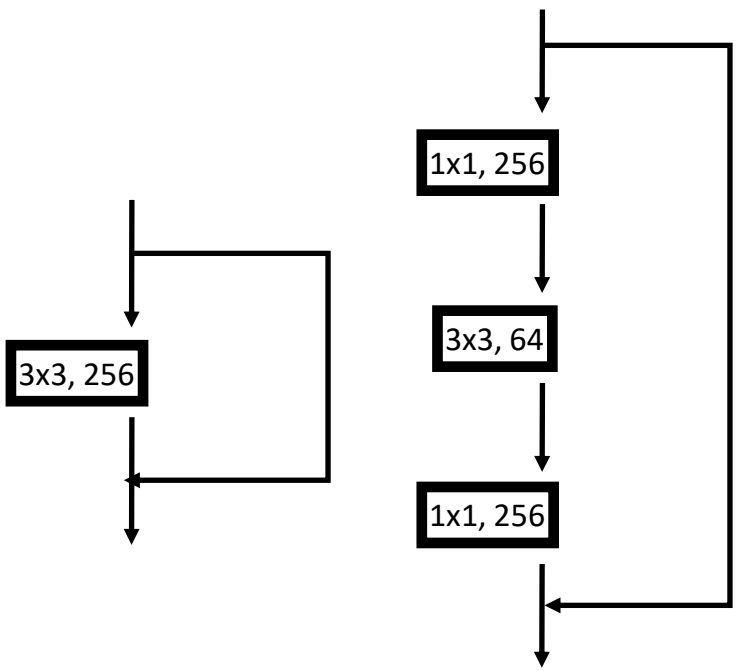


A Building Block of Residual Network



Res-Net, bottleneck layer

- Since the network is very deep now, the complexity is high.
- A bottleneck design is used to reduce the complexity



- ResNets greatly improved accuracy on ImageNet
- ResNet-152 still has lower complexity than VGG-19 !!

method	top-1 err.	top-5 err.
VGG [40] (ILSVRC'14)	-	8.43†
GoogLeNet [43] (ILSVRC'14)	-	7.89
VGG [40] (v5)	24.4	7.1
PReLU-net [12]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	**19.38**	**4.49**

10-Crop Testing + Fully Conv with Multiple Scale Results

Summary

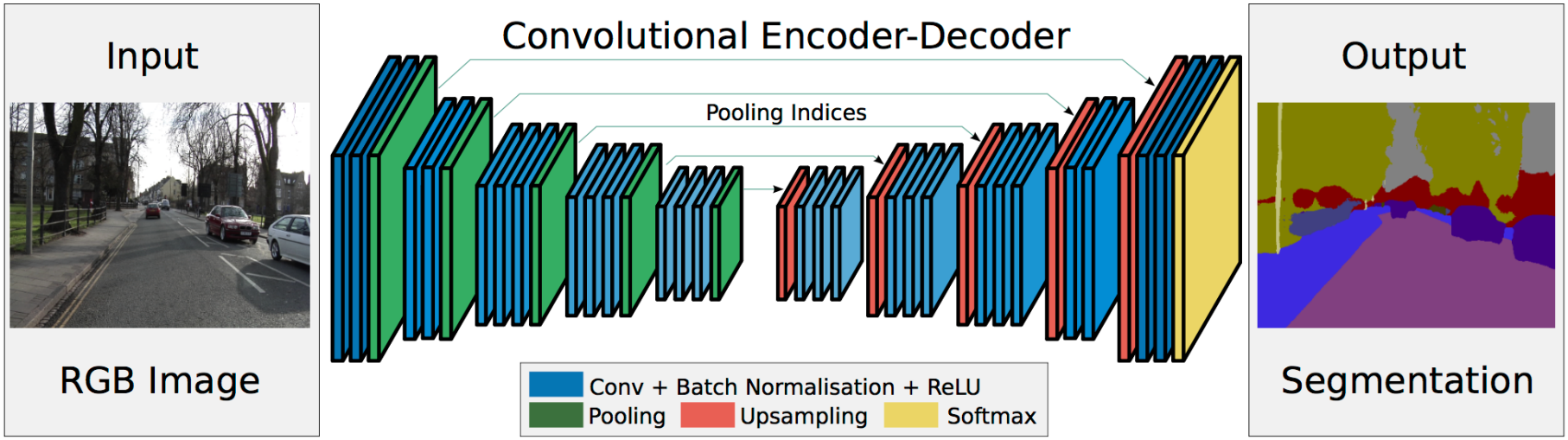
- Convolutional Neural Networks “filter” feature maps (e.g., multichannel “images”).
- Architectural choices, filter sizes, skip connections (residual networks), network depth lead to large differences in performance.

Overview

- Mini-Batch Gradient Descent (SGD)
- Optimisation (momentum, learning rate, regularization, augmentation)
- Convolutional Neural Networks
  - Convolution and max-pooling
  - AlexNet, VGG, ResNet, MobileNet, Fully Convolutional
- Other topics
  - Transfer learning, Multi-task learning, Unsupervised learning

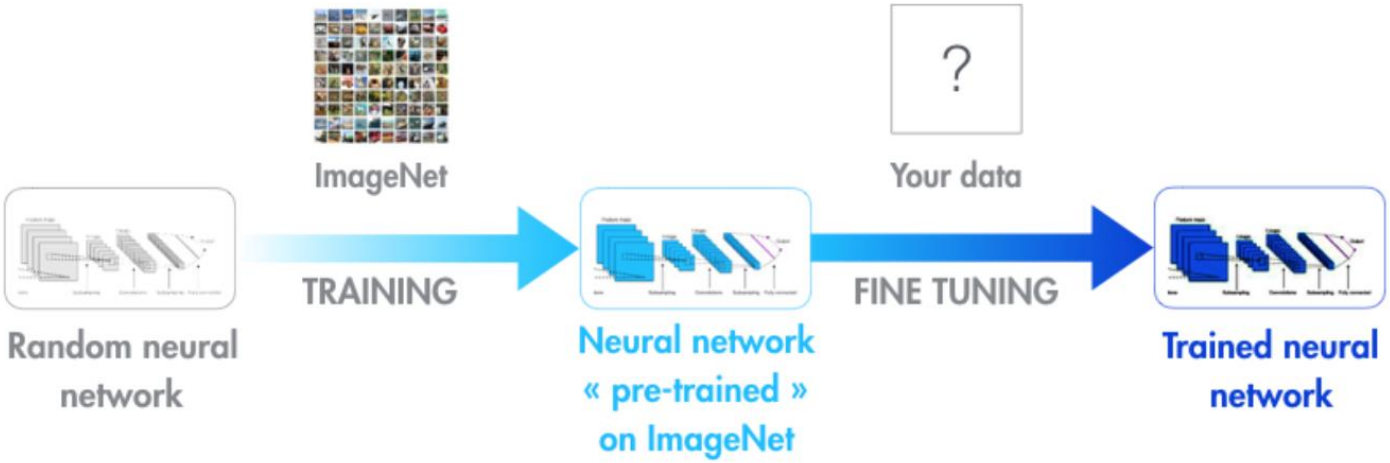
Fully Convolutional Architectures

- From image to label map
- No Fully Connected layers exist
- For example assume we want to label every pixel with an object class label (semantic segmentation)
- The loss is applied in the end at each spatial location



Transfer Learning

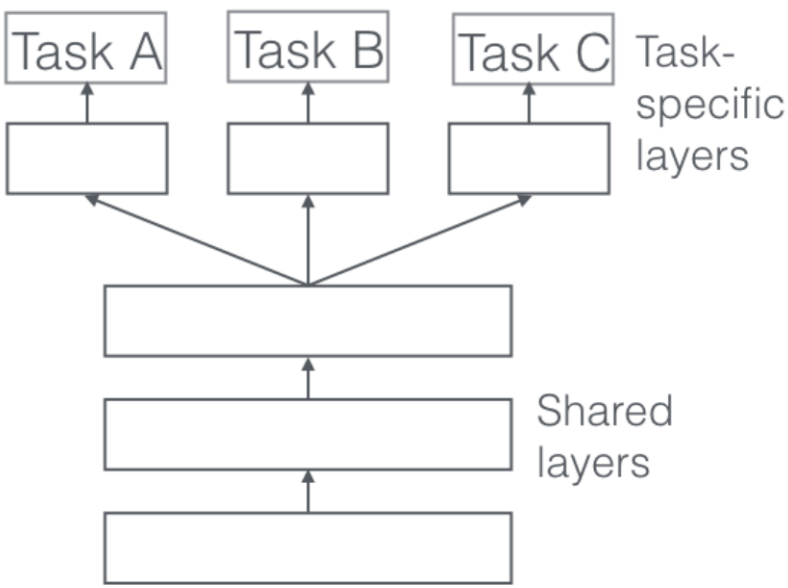
- Pre-train a network on one dataset (usually large scale, with lot of classes)
  - ImageNet is the best candidate
- Use the weights as initialisation for a new dataset
  - If the new dataset is small then this works much better than initialising from scratch

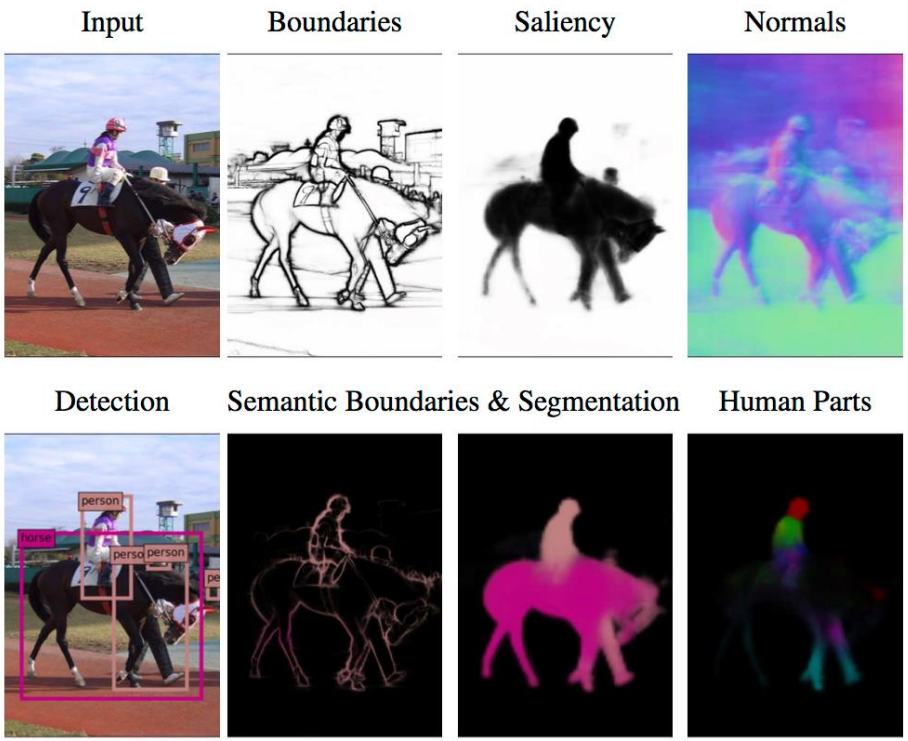


https://medium.com/kansas-city-machine-learning-artificial-intelligen/an-introduction-to-transfer-learning-in-machine-learning-7efd104b6026

Multi-task learning

- A train a single network to perform several tasks
  - A single network is shared for all tasks
  - Specialised networks (called heads) for each task
  - In some cases multi-task learning improves accuracy (with lots of data better train a single network)





Unsupervised Learning

- ImageNet pretraining works well but we want to do better
- Collecting and labelling a bigger dataset very expensive (does not scale up)
- Solution: pretrain a network on large amounts of data without labels!
- We need to define a task that does not require human annotation. E.g.:

Image jigsaw solving

Impainting

Rotation estimation

Summary

- Recent advances in CNNs
  - Fully convolutional Networks (segmentation, depth estimation, …)
  - Multitasking
  - Transfer Learning
  - Unsupervised Learning